Prerequisites and objectives

- Prerequisites are C++ functions and loops, basic mathematical induction, logarithms, and geometric series.
- Express inputs, outputs, and error conditions as contracts. Explain correctness with an invariant.
- Specify the input size and cost model before distinguishing worst-case, expected, and amortized costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Specify the problem before the program

- "Find a key in a sorted array" leaves duplicates, unsuccessful searches, and empty arrays undefined.
- Here `lower_bound(A,x)` returns the smallest $i \in [0,n]$ such that $i=n$ or $A[i] \geq x$. The precondition is that A is nondecreasing.
- The postcondition is $\forall j<i, A[j]<x$ and $\forall j \geq i, A[j] \geq x$, with both quantifiers restricted to valid array indices. Returning n is a valid insertion position.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search specification

1. Initialize `l=0,r=n`. Candidate insertion positions form the closed interval `[l,r]`.
2. While `l<r`, set `m=l+⌊(r-l)/2⌋`.
3. If `A[m]<x`, set `l=m+1`. Otherwise set `r=m`.
4. Return `l`. Every inspected index satisfies `m<n`. The algorithm never reads `A[n]`.

Distinguish candidate positions from the unclassified element interval `[l,r)`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The invariant connects specification and implementation

- The loop invariant is $0 \le l \le r \le n$, $\forall j < l, A[j] < x$, and $\forall j \ge r, A[j] \ge x$, for valid indices j.
- If $A[m] < x$, sorted order excludes every $j \le m$. In the other branch, every $j \ge m$ belongs to the right classified region.
- At termination $l = r$. The classified regions cover all elements and give exactly the postcondition.
- The termination measure $r - l$ is a nonnegative integer that strictly decreases each iteration. Partial correctness plus termination gives total correctness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked trace and boundary cases

Search `A=[2,4,4,9,13]` for `x=4`.

Iteration	l	r	m	Comparison and update
1	0	5	2	$4 \geq 4$, r=2
2	0	2	1	$4 \geq 4$, r=1
3	0	1	0	$2 < 4$, l=1

The result is 1. An empty array returns 0 immediately. Searching for `x=14` returns 5. These cases check the contract rather than merely finding some occurrence of 4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary-search contract and boundary traces

```
lower_bound(A,x):
    l=0; r=len(A)
    while l<r:
        m=l+(r-l)//2
        if A[m]<x: l=m+1
        else: r=m
    return l
```

Require nondecreasing A and a total-order comparison. For A=[], initialization l=r=0 returns 0 without a read. For A=[2,4,4,9,13], x=14, states (l, r, m) are (0,5,2)→(3,5,4)→(5,5, none), returning 5. With x=1 they are (0,5,2)→(0,2,1)→(0,1,0)→(0,0, none), returning 0. Neither reads A[5].

The duplicate-key contract requires the first not-less position, not an arbitrary equal position. Unsorted data or a nontransitive comparator invalidates half-interval elimination. The tight comparison count is Θ(log(n+1)); worst-case comparisons of length-L keys add O(L) each.

Data abstraction and representation invariants

- An abstract data type, or ADT, defines observable values and operations without fixing storage. A set may require membership, insertion, and deletion.
- A representation invariant restricts valid implementation states. A dynamic array may require $0 \le \texttt{size} \le \texttt{capacity}$ and valid elements in its first `size` positions.
- An abstraction function maps each valid implementation state to an abstract value. A correct operation preserves both the representation invariant and the ADT semantics.
- A sorted array, hash table, or balanced tree can implement the same set ADT. Their costs and supported order operations differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Time depends on the computation model

- Let `n` be the array length. If integer comparison and indexed access each cost `O(1)`, binary search takes `O(log(n+1))` worst-case time.
- For string keys of length at most `L`, one lexicographic comparison can cost `O(L)` in the worst case. It is not automatically constant time.
- For disk-resident data, random access can dominate. RAM operations and page I/Os are different metrics.
- Cache behavior, allocation, and compilation affect measured time. An asymptotic growth rate does not determine milliseconds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quantifiers in O, Ω, and Θ

- $f \in O(g)$ means that constants $c > 0, n_0$ exist such that $0 \leq f(n) \leq cg(n)$ for every $n \geq n_0$.
- $f \in \Omega(g)$ gives the corresponding lower bound. $f \in \Theta(g)$ means that both hold.
- $3n+8 \in \Theta(n)$. The statement $3n+8 \in O(n^2)$ is also true but is not tight.
- Worst-case time is itself a cost function, for example $W(n) = \max_{\{|x|=n\}} T(x)$. The symbol O alone means neither worst case nor average case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derive sums from loops

- Performing i constant-time operations for each $i=1..n$ costs $\Sigma i=n(n+1)/2=\Theta(n^2)$.
- Starting at $i=1$ and repeatedly doubling until $i \geq n$ takes $\lceil \log_2 n \rceil$ iterations for $n \geq 1$, not $n/2$.
- Nested-loop depth does not determine time. Inner-loop lengths $n, n/2, n/4, \ldots$ sum to less than $2n$.
- If the output contains n^2 values, any algorithm that outputs each value needs $\Omega(n^2)$ output operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive time and recursive space

- Merge sort satisfies $T(n)=2T(n/2)+\Theta(n)$. Each level performs $\Theta(n)$ total work, with $\Theta(\log n)$ levels.
- The call stack follows one active path of depth $\Theta(\log n)$. The number of nodes in the entire recursion tree is not the number of simultaneously live frames.
- Sharing one n-element merge buffer uses $\Theta(n)$ auxiliary array space. Including the stack still gives $\Theta(n)$ auxiliary space.
- Separate input, output, and auxiliary space. An in-place claim must state whether it excludes the recursion stack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worst-case, expected, and amortized costs

Analysis	What is aggregated	Example
Worst case	Maximum over all inputs of a fixed size	Binary search $\Theta(\log n)$ for $n \geq 2$
Expected	A stated probability space	Random-pivot quicksort expected $\Theta(n \log n)$
Amortized	A whole legal operation sequence	Doubling-array append amortized $\Theta(1)$

Amortized analysis requires no random input. Expectation may average over the algorithm's own random choices without assuming uniformly distributed input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Aggregate proof for a doubling array

- When capacity c is exhausted, grow it to $2c$. Assume a constant upper bound on moving one element and an initial capacity of at least 1.
- In n appends from an empty array, new-element writes total n. For $n \geq 2$, moves are at most $1+2+4+\ldots+2^{\lfloor \log_2(n-1) \rfloor} < 2n$. For $n \leq 1$, no expansion is needed from capacity 1.
- Total cost is $O(n)$, so append costs amortized $O(1)$. A particular expansion can still cost $O(n)$.
- Growing by only one slot each time moves $1+2+\ldots+(n-1)=O(n^2)$ elements. The growth rule changes the amortized cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic-array potential versus actual work

```
append(T value):       // by-value snapshot exists before this body
    if size==capacity:
        allocate max(1,2*capacity) slots
        relocate existing elements; commit storage
    construct from independent value at storage[size]
    size=size+1
```

Start empty at capacity 1, charging one unit for a new write and one per moved value. For reachable nonempty states with $c \le 2n$, use $\Phi = 2n - c$; handle the initial $\Phi = -1$ with a constant startup charge. Without growth, actual cost is 1 and $\Delta\Phi = 2$, giving amortized 3. At full $n = c$, growth to $2c$ followed by insertion costs $c+1$; new potential 2 minus old potential c again gives amortized 3. Linear allocation work changes only the coefficient.

States (n,c,actual cost) are (1,1,1),(2,2,2),(3,4,3),(4,4,1),(5,8,5). Actual cost 5 does not contradict constant amortization. Shrinking exactly to size after every pop can cause repeated relocation under alternating updates; that policy needs a separate analysis and separated resize thresholds.

Dynamic-array potential versus actual work (continued)

Append(T value) explicitly constructs an independent by-value parameter before entering the function body; it is not a const T& parameter. Thus append(A[0]) snapshots the argument before full-capacity growth invalidates old storage. At capacity1 with[7], snapshot7, relocate the original7, release old storage, and construct the new element from the independent value, producing[7,7]. A const T& interface that grows first instead leaves a reference to old A[0] dangling.

This section assumes copy-constructible T. Argument-snapshot cost C_arg is separate: the amortized3 above counts only its stated storage writes and relocations. Charging one additional unit per snapshot gives amortized at most4. Copying a length-L string can cost Θ(L), which must be added rather than called constant. Failed parameter construction happens before the body modifies the array.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A counter potential for nonuniform write costs

```
increment(A):
    require A is not the maximum k-bit value
    i=0
    while A[i]==1: A[i]=0; i+=1
    A[i]=1
// A[0] is the least significant bit.
// Read cost: 1; write cost at i: c_i=(3/2)^i.
// Phi(A)=sum_{i=0}^{k-1} (1+3*(3/2)^i)*A[i].
```

Amortization stores predictable future work in the current state. This model charges one unit per bit read and $c_i = (3/2)^i$ per write, excluding other fixed instructions. Start at zero and stop before overflow. If the low r bits are one, an increment reads $r + 1$ bits and writes bits $0, \ldots, r$. Its actual cost is

$$C_r = (r + 1) + \sum_{i=0}^{r} (3/2)^i = r - 1 + 3(3/2)^r.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A counter potential for nonuniform write costs (continued)

Use the explicit nonnegative potential $\Phi = \sum_{i=0}^{k-1} a_i A[i]$, with $a_i = 1 + 3(3/2)^i$. No coefficient remains undetermined. Clearing the low r ones and setting bit r gives

$$\Delta\Phi = a_r - \sum_{i=0}^{r-1} a_i = 7 - r - 3(3/2)^r.$$

Thus $C_r + \Delta\Phi = 6$, independent of k and r. Since $\Phi_0 = 0$, the total actual cost of T operations is $6T - \Phi_T \le 6T$.

With the low bit on the right, 000→001 has $C = 2, \Delta\Phi = 4$; 001→010 has $C = 9/2, \Delta\Phi = 3/2$; 010→011 has $C = 2, \Delta\Phi = 4$; and 011→100 has $C = 31/4, \Delta\Phi = -7/4$. Each sum is 6. The four operations cost $65/4$ and leave potential $31/4$, summing to 24. Counting unweighted one bits would not store enough credit for expensive high-bit writes.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Deriving coefficients and the cost-growth boundary

```
// General processed-bit charge: q_i=b+alpha^i, 0<alpha<2, b>=0.
K=sum_{j=0}^infinity q_j/2^j = 2*b+2/(2-alpha)
a_i=2^i*sum_{j=i+1}^infinity q_j/2^j
   =b+alpha^(i+1)/(2-alpha)
Phi(A)=sum_i a_i*A[i]
```

Let the full charge for a processed bit be $q_i = b + \alpha^i$, where b explicitly includes reads and the selected fixed control work. Requiring the same amortized cost K for every carry length r gives

$$a_r = K - \sum_{i=0}^{r} q_i + \sum_{i=0}^{r-1} a_i = 2^r \left(K - \sum_{j=0}^{r} q_j / 2^j \right).$$

Choose $K = \sum_{j=0}^{\infty} q_j / 2^j$. The remaining tail is nonnegative, so

$$K = 2b + \frac{2}{2-\alpha}, \qquad a_i = 2^i \sum_{j=i+1}^{\infty} \frac{q_j}{2^j} = b + \frac{\alpha^{i+1}}{2-\alpha}.$$

Deriving coefficients and the cost-growth boundary (continued)

This derives the coefficients from future costs instead of guessing a potential and leaving its constants unspecified.

For $\alpha < 2$, the series converges and all coefficients are computable real numbers. The preceding case $b = 1, \alpha = 3/2$ gives $K = 6$ and $a_i = 1 + 3(3/2)^i$. Charging exactly three additional control units per processed bit changes b to 4, giving $K = 12$ and $a_i = 4 + 3(3/2)^i$. An arbitrary initial configuration adds its initial potential to the total-cost bound, so the guarantee is not independent of the starting state.

At the boundary $\alpha = 2$, incrementing from zero to all ones takes $2^k - 1$ operations. Bit i is written $2^{k-i} - 1$ times. Total write cost is

$$\sum_{i=0}^{k-1} 2^i (2^{k-i} - 1) = k2^k - (2^k - 1),$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Deriving coefficients and the cost-growth boundary (continued)

so the average cost is $\Theta(k)$. No total-sequence constant uniform in k exists. This follows from actual required work, not merely the failure of one candidate potential. A fixed k gives a finite constant, but this asymptotic claim must hold as k varies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and teaching boundaries

- The binary-search trace, contracts, sums, and doubling proof are independently designed explanations.
- The C++ working draft vector overview specifies contiguous storage and amortized end-operation costs. It does not require a fixed doubling factor.
- The MIT 6.006 2020 lecture index, L1–L2, provides course context.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study